VEL TECH RANGARAJAN DR. SAGUNTHALA R&D INSTITUTE OF SCIENCE AND TECHNOLOGY
Chennai, India · Office of International Relations · Value Added Course

Digital Twin with Python

VTR UGE 21

DAY 1 · MODULE 1

Concept and Architecture

What separates a digital twin from a simulation, measured on one battery pack, and the architecture that holds it.

Prof. Dr. Utku Kose
Full Professor, Department of Computer Engineering, Süleyman Demirel University, Isparta, Türkiye
Founding Director, AI Application and Research Center (YAZEM), Süleyman Demirel University
Head of the Computer Science Division, Department of Computer Engineering, Süleyman Demirel University
Additional affiliations: University of North Dakota (USA), Universidad Panamericana (Mexico City, Mexico), Vel Tech University (Chennai, India)
IEEE Senior Member, ACM Professional Member

ORCID 0000-0002-9652-6415 · utkukose.com · github.com/utkukose
utkukose@sdu.edu.tr · utku.kose@und.edu · ukose@up.edu.mx · utkukose@gmail.com

Overview

A digital twin is a virtual counterpart of one specific physical asset, kept in step with it by data and able to act on it [1, 2]. The term is also used for models drawn with computer-aided design (CAD) software, dashboards and simulations, which lack exactly the parts that make a twin useful and difficult. This day fixes the meaning of the term, introduces the asset of the course, a 48 V lithium-ion battery pack, and shows on that pack what each data flow between the asset and its virtual counterpart adds. It then sets up a reference architecture of five components [3, 4] and explains why twins are built and tested against a simulated asset before hardware exists [5].

Estimated study time: 6 to 8 hours.

Learning outcomes

By the end of the day, students are expected to define a digital twin by the data flows between the physical and the virtual entity and distinguish it from a digital model and a digital shadow, to describe the states, inputs, parameters and time scales of the battery pack, to measure the error and the operational benefit of each level of maturity, to choose a synchronisation interval by experiment rather than by intuition, to map an implementation to a five-component reference architecture and to grade a twin against scenarios with a known ground truth. In Python, students are expected to use values, variables, lists, loops, decisions, functions and dictionaries to simulate and control the pack temperature step by step.

Python in this day

The Colab notebook opens with Python step 1: Values, variables, lists and decisions, right after its setup section. It covers values and types, f-strings, `if` statements, lists and `for` loops, functions, dictionaries and classes, applied to the temperature of the battery pack: A fan rule, a list of readings, five steps of the thermal model computed by hand and a pack object that repeats them [6]. After the step, the notebook continues with the full twin. Students who have never programmed should first open the start-here notebook of the course.

How these notes point to the Python code. Every method of this lecture is also written as Python code in the Colab notebook of the day, the file `NB01_concept_and_architecture.ipynb`. A Colab notebook is a document of text and Python code that runs in the web browser, with nothing to install. It opens with the button Open in Colab on the day overview page of the course, https://github.com/utkukose/digital-twin-veltech-NB-lecture/tree/main/days/day-01. The code of the notebook is divided into numbered sections. In these notes, each box titled *Python code in the Colab notebook, Section N* names the section whose code carries out what the text above the box explains, and says what to run and what to look at there. The animations that the notes mention are interactive figures on the lecture page of the day, https://utkukose.github.io/digital-twin-veltech-NB-lecture/days/day-01/lecture.html. The interactive lab, with the practice parts and the self-assessment of the day, is at https://utkukose.github.io/digital-twin-veltech-NB-lecture/days/day-01/lab.html. Each box titled *Interactive lab, Part X* opens one part of the lab. The part ends with a link back to the lecture page, and the notebook is always opened from the boxes of the lecture.

Python step 1: Values, variables, lists and decisions

A notebook is a sequence of cells. Text cells explain, and code cells hold Python instructions that run when the cell is executed with Shift+Enter, with the output shown directly below the cell [10]. This step

works with the temperature of the battery pack: It computes five steps of the thermal model by hand and ends with the pack as an object of a class [6].

Values and variables

```
soc = 0.82              # state of charge, 0 to 1
temperature_C = 41.3    # cell temperature in degrees Celsius
current_A = 40.0        # current drawn by the load
chemistry = "lithium-ion"
print(type(soc), type(chemistry))
print("SoC:", soc, "temperature:", temperature_C)
```

Output

```
<class 'float'> <class 'str'>
SoC: 0.82 temperature: 41.3
```

A variable is a name bound to a value with `=`. Numbers with a decimal point are floats, text in quotation marks is a string, and `True` and `False` are booleans. Units belong in the names or in a comment after `#`, because a number without a unit is a source of errors in every engineering program.

A decision is an if statement

```
fan_on_C = 42.0
if temperature_C > fan_on_C:
    fan = "on"
else:
    fan = "off"
print(f"Temperature {temperature_C:.1f} °C, threshold {fan_on_C:.1f} °C, fan {fan}")
```

Output

```
Temperature 41.3 °C, threshold 42.0 °C, fan off
```

An `if` statement runs the indented block below it only when its condition is true, and `else` covers the remaining case. An f-string, written with `f` before the quotation mark, inserts the value of each expression in braces, and `:.1f` prints one decimal. This rule is the simplest controller of the pack.

Lists and loops

```
readings = [36.0, 39.4, 42.1, 44.8, 46.9, 48.2]
hot = 0
for minute, t in zip(range(0, 60, 10), readings):
    status = "over limit" if t > 46 else "ok"
    hot = hot + (t > 46)
    print(f"minute {minute:2d}: {t:4.1f} °C  {status}")
print("readings over 46 °C:", hot)
```

Output

```
minute  0: 36.0 °C  ok
minute 10: 39.4 °C  ok
minute 20: 42.1 °C  ok
minute 30: 44.8 °C  ok
minute 40: 46.9 °C  over limit
minute 50: 48.2 °C  over limit
readings over 46 °C: 2
```

A list holds values in order between square brackets. `range(0, 60, 10)` produces 0, 10, 20 and so on, and `zip` walks two sequences in step: `for minute, t in ...` gives the two values of each step the names `minute` and `t`. The one-line form `a if condition else b` chooses between two values, and adding a boolean to a number counts it as 1 or 0. In the f-string, `:2d` and `:4.1f` reserve two and four places, so that the columns line up.

A simulation is a loop

```
T, T_amb = 36.0, 35.0                 # pack and ambient temperature
R, m_cp, hA = 0.045, 9000.0, 2.2      # ohm, J/K, W/K
dt = 60.0                             # one minute in seconds
for minute in range(1, 6):
    heat_in = current_A ** 2 * R
    heat_out = hA * (T - T_amb)
    T = T + dt * (heat_in - heat_out) / m_cp
    print(f"after {minute} min: {T:.2f} °C")
```

Output

```
after 1 min: 36.47 °C
after 2 min: 36.92 °C
after 3 min: 37.38 °C
after 4 min: 37.82 °C
after 5 min: 38.26 °C
```

Each pass of the loop applies the energy balance of the lecture for one minute: The new temperature is the old one plus the time step times the rate of change. This is the explicit Euler method, and Day 2 measures exactly how accurate it is. The operator `**` raises to a power.

Functions and dictionaries

```
def fan_duty(T, on_C=42.0, span_C=6.0):
    """Proportional fan command between 0 and 1."""
    return min(1.0, max(0.0, (T - on_C) / span_C))

reading = {"T": 47.3, "SoC": 0.64, "I": 41.0}
reading["fan"] = fan_duty(reading["T"])
for key, value in reading.items():
    print(f"{key:4s} = {value:.2f}")
```

Output

```
T    = 47.30
SoC  = 0.64
I    = 41.00
fan  = 0.88
```

`def` defines a function with parameters, default values and a `return` statement, and the text in triple quotation marks documents it. `min` and `max` clip the command to the range from 0 to 1. A dictionary maps keys to values: `reading["T"]` looks a value up, assigning to a new key adds an entry, and `items` returns key and value pairs for a loop. A sensor message is naturally a dictionary.

A pack as an object

```
class Pack:
    """A pack that keeps its temperature and advances it one time step at a time."""
    def __init__(self, T, current_A):
        self.T = T                      # temperature in degrees Celsius
        self.current_A = current_A      # current drawn by the load
    def step(self, dt):
        heat_in = self.current_A ** 2 * R
        heat_out = hA * (self.T - T_amb)
        self.T = self.T + dt * (heat_in - heat_out) / m_cp

pack = Pack(36.0, current_A)
for minute in range(1, 6):
    pack.step(dt)
print(f"after 5 min: {pack.T:.2f} °C")
```

Output

```
after 5 min: 38.26 °C
```

A class bundles values with the functions that use them. `Pack(36.0, current_A)` creates one object of the class: Python runs `__init__`, which stores the two values on `self`, the object itself. Values stored on `self` are called attributes, and functions defined inside the class are called methods. Every method receives the object as its first argument `self`, so `step` reads and changes the temperature of its own pack, and `pack.step(dt)` calls it. `pack.T` reads an attribute from outside. The object repeats the five steps of the loop above, with `R`, `hA`, `T_amb` and `m_cp` from that cell, and ends at the same temperature. `@dataclass` in front of a class, as in front of `PackParams` in the setup cell, writes such an `__init__` automatically from a list of attributes with default values. The notebook later builds the five parts of the twin as classes of this kind.

Quick check. What does `fan_duty(39.0)` return, and why?

What is a digital twin?

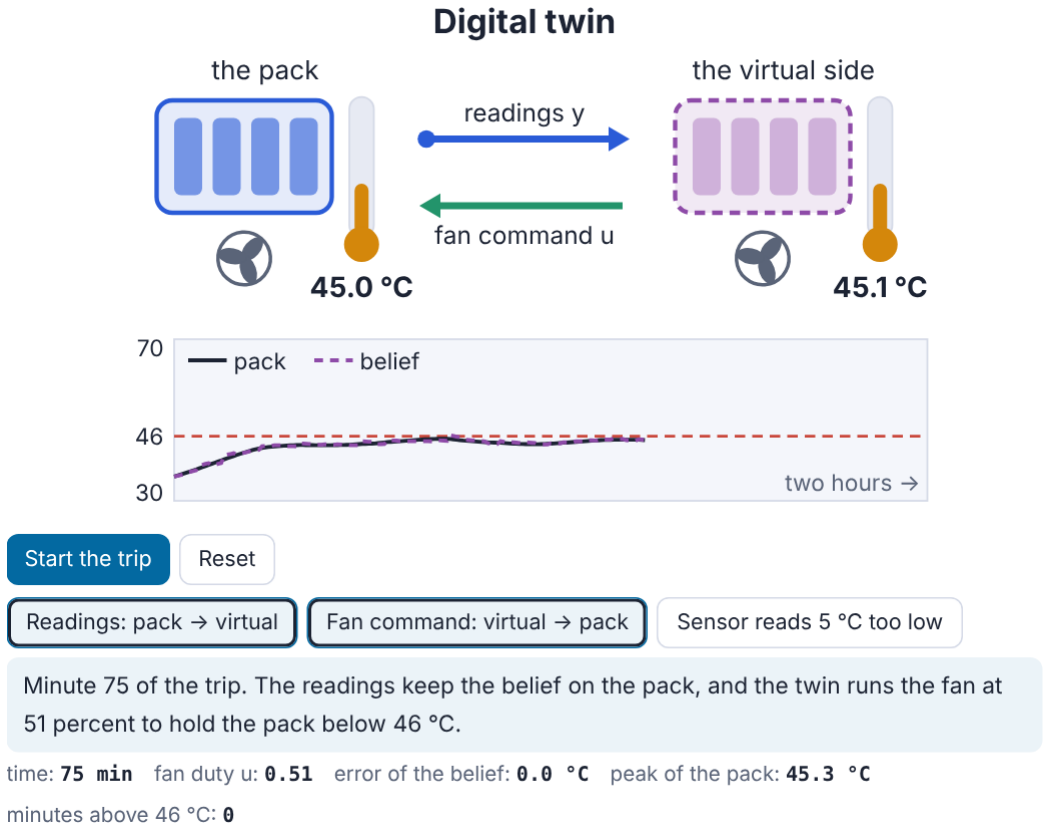
A digital twin always belongs to one physical thing, which is called its asset: a machine, a vehicle, a building, a battery pack or, in medicine, a patient. Reference texts also call the asset the physical entity, and the program that represents it the virtual entity or virtual counterpart. The numbers that describe the condition of the asset at one moment, such as its temperature and its charge, form its state.

Grieves and Vickers introduced the digital twin as a virtual representation that stays linked to one specific physical product throughout its life [1]. Kritzinger and colleagues made the idea checkable by looking at the data flows between the two [2]. A digital model exchanges no data with the asset automatically. A digital shadow receives data from the asset automatically, so its state follows the real one. A digital twin also sends data back automatically and changes the asset, for example by switching on a fan.

Fixed symbols make these data flows precise, and the whole course uses them. The state of the asset at time t is written $x(t)$. The virtual side cannot see it and keeps an estimate, written with a hat, $\hat{x}(t)$, which the course calls the belief. A sensor delivers measurements, also called readings: y_k is reading number k. The value that the virtual side sends back is the input u, in this course the setting of a cooling fan. The first data flow carries y_k from the asset to the virtual side, and the second carries u back. Diagrams draw each flow as an arrow, so the lecture speaks of the first and the second arrow.

Level	From asset to model, y_k	From model to asset, u	Example
Digital model	by hand	by hand	a simulation run with the values of the manufacturer's datasheet
Digital shadow	automatic	by hand	a monitoring screen that follows the pack temperature
Digital twin	automatic	automatic	a system that follows the pack and switches its fan

The three levels are also called levels of maturity, and their sequence the maturity ladder: Each step up makes one more arrow automatic. The level is decided by what a system does and not by its name. A detailed CAD drawing and a live dashboard are often presented as digital twins, yet by their data flows the first is a digital model and the second a digital shadow [2].



Interactive scene on the lecture page: Two arrows. The battery pack of the course and its virtual counterpart, joined by the two arrows of this section. With both arrows off, the virtual side is a digital model that runs on the datasheet values. The readings make it a digital shadow, and the fan command makes it a digital twin. Each trip lasts two hours, with a current of about 40 A and air at 35 °C, computed in the browser with the model of the next section. A third switch makes the sensor read 5 °C too low.

> **In everyday life: model, shadow and twin**
>
> A printed bus timetable is a model of a bus line: It says where each bus should be and never learns where it is. A live map in a transport app, fed with the positions of the buses, follows the line like a shadow: A passenger sees that a bus is late but cannot change it. A traffic-light system that gives a late bus a longer green phase reads the position of the bus and acts on its journey, as a twin acts on its asset.

A navigation app shows the three levels on a single trip. A car drives 10 km to its destination at 1 km per minute, so the trip takes 10 minutes on a free road. At minute 3 a jam forms between km 4 and km 6, where the traffic moves at 0.2 km per minute, which is 12 km/h. A detour of 8 km leaves the road at km 3.5 and leads to the destination at 1 km per minute.

Animation on the lecture page: A navigation app as a digital model, a digital shadow and a digital twin of one car on one trip. Choose a level and play the trip, then compare the arrival that the app announces with the arrival of the car. For the shadow, a box lets the driver read the advice and turn by hand.

As a digital model the app announces minute 10 for the whole trip, because it never hears from the car or the road. The car reaches the jam at minute 4, needs 2 / 0.2 = 10 minutes for its 2 km and covers the last 4 km in 4 minutes, so it arrives at minute 4 + 10 + 4 = 18. As a digital shadow the app follows the car and the traffic, and from minute 3 it announces minute 18. The car takes the detour only if the driver reads the advice and turns by hand. As a digital twin the app also sends the new route to the car, which turns at km 3.5 at minute 3.5 and arrives at minute 3.5 + 8 = 11.5, which is 18 − 11.5 = 6.5 minutes earlier. The first arrow makes the announcement right, and the second arrow makes the trip shorter.

> Check your understanding (1). A dashboard shows live temperatures from a machine, and an operator adjusts the machine by hand. Which level is it?
>
> A. Digital model
>
> B. Digital shadow
>
> C. Digital twin
>
> D. None

> **Interactive lab, Part A.**
>
> Six systems from industry, buildings and medicine, each to be classified as a digital model, a digital shadow or a digital twin by its data flows. Part A opens at https://utkukose.github.io/digital-twin-veltech-NB-lecture/days/day-01/lab.html#pa.

The asset of the course

Every day of the course works on the same asset: a 48 V lithium-ion battery pack of the kind used in light electric vehicles and energy storage. Its state consists of the charge, the cell temperature, the usable capacity and the internal resistance. The current drawn by the load heats the pack, the air around it cools it, and a fan can increase the cooling.

The twin of the pack, the project of the course

Building the digital twin of this pack is the project of the course, and each day adds one layer to it. The pack itself is simulated, so its true state is always known and every part of the twin can be graded against it.

Day 1 states the model of the pack and climbs the maturity ladder from a digital model to a digital shadow and a digital twin that switches the fan. Day 2 tests the simulation core that every part of the twin calls, and it adds a charging depot and a fleet of vehicles with such packs. Day 3 turns the telemetry of the pack into a clean time series, calibrates the parameters of the model on measurements and replaces the simple correction of the belief by a Kalman filter. Day 4 adds the services of the intelligence layer: fast surrogate models, a hybrid model, fault detection, the remaining useful life and a controller that plans the fan. Day 5 assembles the parts into one twin with a dashboard, a frozen configuration and an event log, and it collects the evidence for how far the twin can be trusted.

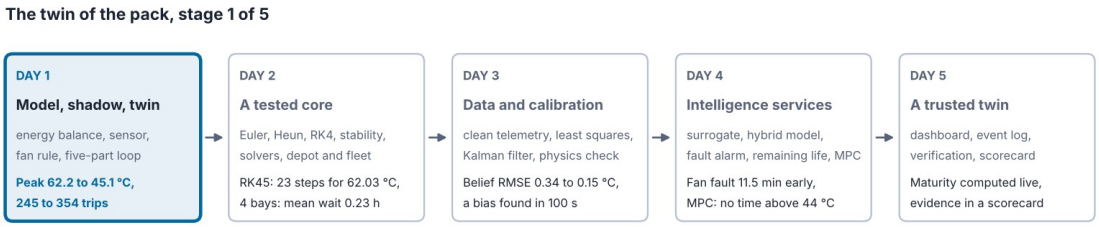


Figure. The five stages of the project through the week. Today the pack gets its model, and the virtual side climbs from a digital model to a twin that switches the fan.

A model of the asset is a rule for how the state changes. It gives the rate of change of the state, its change per second, from the present state, the input and the parameters:

$$\frac{dx}{dt} = f(x, u, \theta)$$

The left side is the rate of change of x, the slope of the state when it is plotted against time, also written $\dot{x}$. The parameters θ, the Greek letter theta, are numbers that stay fixed for one asset, such as its heat capacity. An equation of this kind is called a differential equation. For the pack, the state has four parts, x = (SoC, T, Q, R), and the fan duty u is the input that the twin can set. Two further inputs come from outside: the current I drawn by the load and the temperature of the surrounding air. The two tables list every quantity with its name in the Colab notebook of the day.

Kind	Symbol	In the code	Meaning	Unit and value
State	SoC	`SoC`	State of charge: the share of the usable capacity that is still in the pack, from 0 for empty to 1 for full	no unit, 0.95 at the start
State	T	`T`	Temperature of the cells	degrees Celsius (°C), 36 at the start
State	Q	`Q`	Usable capacity: the charge that the full pack can deliver	ampere-hours (Ah), 60 when new
State	R	`R`	Internal resistance of the pack	ohm, 0.045 when new

Kind	Symbol	In the code	Meaning	Unit and value
Input	I	`I`	Current drawn by the load, positive while the pack is discharged	ampere (A), between 34 and 46 on the highway trip of the day
Input	u	`u`	Fan duty: the setting of the cooling fan, from 0 for off to 1 for full speed	no unit
Input	T_{amb}	`T_amb`	Ambient temperature: the temperature of the air around the pack	°C, 35 on the trips of the day

Parameter	In the code	Meaning	Value
Q_0	`Q0`	Capacity of the new pack	60 Ah
R_0	`R0`	Internal resistance of the new pack	0.045 ohm
$m\,c_p$	`m_cp`	Thermal mass, also called heat capacity: the energy that warms the pack by one degree. It is the mass m times the specific heat c_p, and the model needs only the product	9000 joules per kelvin (J/K)
hA_0	`hA0`	Passive conductance: the heat that leaves the pack per second for each degree of difference to the air, with the fan off. It is a heat transfer coefficient h times a surface A, stored as one number	2.2 watts per kelvin (W/K)
hA_1	`hA1`	Additional conductance at full fan speed	12 W/K
k_{age}	`k_age`	Ageing constant: the capacity lost per ampere-second of current at the reference temperature	5.5×10^{-8} Ah
E_a / R_g	`Ea_R`	Sensitivity of the ageing to temperature	3800 kelvin (K)
T_{ref}	`T_ref`	Reference temperature of the ageing law	298.15 K, which is 25 °C

A difference of one kelvin equals a difference of one degree Celsius, and one watt is one joule per second. The cells have a limit of 46 degrees, above which the pack counts as overheated. Each part of the state has its own equation.

Charge. The current empties the pack:

$$\frac{d\,\mathrm{SoC}}{dt} = -\frac{I}{3600\,Q}$$

A capacity of Q ampere-hours holds 3600 Q ampere-seconds, and a current of I amperes removes I ampere-seconds every second. The fraction is therefore the share of the capacity that leaves the pack each second, and the minus sign says that the state of charge falls. At 40 A and 60 Ah the pack loses 40 / 60 = 0.67 of its charge per hour and 0.67 / 60 = 0.011 per minute, about 1.1 percentage points.

Temperature. The temperature follows an energy balance: The pack warms when more heat is produced inside it than leaves it.

$$\frac{dT}{dt} = \frac{I^2R - (hA_0 + hA_1u)\,(T - T_{\text{amb}})}{mc_p}$$

The numerator is the balance of two powers in watts. The first, $I^2 R$, is the heat that the current produces in the internal resistance, called ohmic heating: $40^2 \times 0.045 = 1600 \times 0.045 = 72$ W at 40 A. The second is the heat that the air carries away, called convection. It is proportional to the difference between the cell temperature and the ambient temperature, and the factor in front of this difference is the conductance, which the fan raises from hA_0 to $hA_0 + hA_1$. Dividing by the thermal mass $m\,c_p$ turns watts into degrees per second.

> **In everyday life: an energy balance**
>
> A laptop under heavy load warms up, and its fan speeds up when the processor gets hot. Its temperature stops rising when the fan and the case carry away as much heat as the processor produces, which is the steady state of the next paragraphs. A room heated by a radiator in winter follows the same balance: It settles at the temperature at which the heat of the radiator equals the heat lost through walls and windows, and an open window, a larger conductance, lowers that temperature.

> **Worked example: the energy balance at one moment**
>
> The pack is at 50 degrees, the air at 35 degrees, and the current is at its highest value, 46 A. The heating is $46^2 \times 0.045 = 2116 \times 0.045 = 95.2$ W. With the fan off, the conductance is $hA_0 = 2.2$ W/K, and the cooling is $2.2 \times (50 - 35) = 2.2 \times 15 = 33$ W. The difference of $95.2 - 33 = 62.2$ W warms the pack by $62.2 / 9000 = 0.00691$ degrees per second, which is $0.00691 \times 60 = 0.41$ degrees per minute. In this and the following worked examples, intermediate values are rounded, and each step continues with the rounded value.
>
> With the fan at half speed, $u = 0.5$, the conductance is $2.2 + 12 \times 0.5 = 2.2 + 6 = 8.2$ W/K, and the cooling is $8.2 \times 15 = 123$ W. The balance is now $95.2 - 123 = -27.8$ W, the rate is $-27.8 / 9000 = -0.00309$ degrees per second, and $-0.00309 \times 60 = -0.19$: The pack cools by about 0.19 degrees per minute.

When the current and the fan setting stay constant, the solution of this equation is known. The temperature moves towards a final value, the steady state T_{ss}, at which heating and cooling are equal. The time constant τ, the Greek letter tau, says how fast it gets there:

$$T_{\text{ss}} = T_{\text{amb}} + \frac{I^2R}{hA_0 + hA_1u}$$

$$\tau = \frac{mc_p}{hA_0 + hA_1u}$$

$$T(t) = T_{\text{ss}} + (T(0) - T_{\text{ss}})\,e^{-t/\tau}$$

The first line follows from setting the rate of change to zero. In the third line, $T(0)$ is the temperature at the start, and $e^{-t/\tau}$ is the exponential function, which falls from 1 towards 0. After one time constant, 37 percent of the distance to the steady state is left.

Worked example: where the temperature is heading

At 40 A the heating is 72 W. Without the fan the conductance is 2.2 W/K, the steady state is 35 + 72 / 2.2 = 35 + 32.73 = 67.73 degrees, and the time constant is 9000 / 2.2 = 4091 seconds, which is 4091 / 60 = 68 minutes. For a trip that starts at 36 degrees, two hours are 7200 seconds, the exponent is 7200 / 4091 = 1.76, and the exponential factor is $e^{-1.76} = 0.172$. The distance to the steady state at the start is 36 − 67.73 = −31.73 degrees, and after two hours −31.73 × 0.172 = −5.46 degrees of it are left. The temperature is then 67.73 − 5.46 = 62.27 degrees. The simulated trip at the end of this section, in which the current swings around 40 A, ends at 62.0 degrees.

At full fan speed the conductance is 2.2 + 12 × 1 = 14.2 W/K. The steady state falls to 35 + 72 / 14.2 = 35 + 5.07 = 40.07 degrees, about 40.1, and the time constant to 9000 / 14.2 = 634 seconds, which is 634 / 60 = 10.6 minutes, about 11 minutes. A suitable fan setting can therefore hold any temperature between 40.1 and 67.7 degrees, which includes values below the limit.

Ageing. Every ampere that flows wears the cells a little, and heat speeds this up:

$$\frac{dQ}{dt} = -k_{\text{age}}\,|I|\,\exp\left[-\frac{E_a}{R_g}\left(\frac{1}{T_K}-\frac{1}{T_{\text{ref}}}\right)\right]$$

$$\frac{dR}{dt} = 9\times10^{-4}\,R_0\,\frac{Q_0 - Q}{Q_0}$$

In the first line, $|I|$ is the current without its sign, $T_K = T + 273.15$ is the cell temperature in kelvin, and $\exp[z]$ is another way to write e^z. The exponential factor is the Arrhenius law, the usual description of how the speed of a chemical reaction grows with temperature, and a process that follows it is called thermally activated. E_a is the activation energy of the reaction and R_g the gas constant, and the model needs only their ratio, 3800 K. The factor is 1 at the reference temperature of 25 degrees, 1.51 at 35 degrees, 2.23 at 45 degrees and 4.08 at 62 degrees. The second line lets the internal resistance grow as capacity is lost, by 0.2 percent within one trip.

Worked example: the Arrhenius factor at 45 degrees

The cell temperature in kelvin is $T_K = 45 + 273.15 = 318.15$ K. The two reciprocals are 1 / 318.15 = 0.0031432 and 1 / T_{ref} = 1 / 298.15 = 0.0033540 per kelvin, and their difference is 0.0031432 − 0.0033540 = −0.0002108. Multiplied by $-E_a / R_g = -3800$ K, it gives the exponent −3800 × (−0.0002108) = 0.801, and the factor is $e^{0.801} = 2.23$. The pack ages 2.23 times as fast as at 25 degrees.

At 62 degrees the same steps give $T_K = 335.15$ K, 1 / 335.15 = 0.0029837, the difference 0.0029837 − 0.0033540 = −0.0003703, the exponent −3800 × (−0.0003703) = 1.407 and the factor $e^{1.407} = 4.08$. A pack at 62 degrees therefore ages 4.08 / 2.23 = 1.8 times as fast as a pack at 45 degrees.

Worked example: the capacity lost in one trip

At the reference temperature, two hours, which are 7200 seconds, at a mean current of 40 A cost 5.5 × 10^{-8} × 40 × 7200 Ah. Here 5.5 × 10^{-8} × 40 = 2.2 × 10^{-6}, and 2.2 × 10^{-6} × 7200 = 0.0158, so the loss is 0.0158 Ah. On the highway trip simulated at the end of this section, the pack is much warmer, and its Arrhenius factor is 3.1 on average. The loss is therefore 0.0158 × 3.1 = 0.049 Ah, and the simulation gives 0.0489 Ah.

The load. The current as a function of time is called the duty cycle of the load. The trip of this day follows a highway profile:

$$I(t) = 40 + 6 \sin\left(\frac{2\pi t}{1800}\right)$$

The time t is in seconds. The sine swings between −1 and +1 and completes one swing when its argument grows by 2π. The current therefore swings between 34 and 46 A around a mean of 40 A, once every 1800 seconds. Later days also use an urban profile, a stop-and-go pattern between 14 and 40 A that repeats every seven minutes and has a mean of 23.6 A, and an idle profile of 8 A. The duty cycle of the load and the duty of the fan share a word and are different quantities.

Steps in time. A computer cannot follow a rate of change continuously. It advances in time steps of length Δt, with the Greek capital letter delta, and computes the state at the times $t_k = k\,\Delta t$, written x_k. The simplest rule is the explicit Euler method: The new state is the old state plus the time step times the rate of change.

$$x_{k+1} = x_k + \Delta t\, f(x_k, u_k, \theta)$$

The animations of this page use this rule with steps of 5 seconds, and the four assets at the end of the day with steps of 5 to 60 seconds.

> **Worked example: one Euler step**
>
> The pack is at 44 degrees, the current is 50 A, the fan runs at half speed, and the step is 10 seconds. The heating is $50^2 \times 0.045 = 2500 \times 0.045 = 112.5$ W. At half speed the conductance is 8.2 W/K, as in the first worked example, and the cooling is $8.2 \times (44 - 35) = 8.2 \times 9 = 73.8$ W. The rate of change is $(112.5 - 73.8) / 9000 = 38.7 / 9000 = 0.0043$ degrees per second. The step adds $10 \times 0.0043 = 0.043$ degrees, and the new temperature is $44 + 0.043 = 44.043$ degrees.

> **Interactive lab, Part B.**
>
> The formulas of this section solved step by step with numbers: the energy balance, one Euler step, the steady state, the time constant and the exact solution after the step. Sliders set the current, the cell temperature, the fan duty, the air temperature and the time step. Each power and each rate is shown one step at a time, and a figure follows the temperature towards its steady state. Part B opens at https://utkukose.github.io/digital-twin-veltech-NB-lecture/days/day-01/lab.html#pb.

The other simulations of the day use a more accurate rule with the same purpose, the classical Runge-Kutta method of order four, abbreviated RK4. It evaluates the rate of change four times within one step and advances with a weighted mean of the four values:

$$k_1 = f(x_k), \qquad k_2 = f\left(x_k + \tfrac{\Delta t}{2} k_1\right)$$

$$k_3 = f\left(x_k + \tfrac{\Delta t}{2} k_2\right), \qquad k_4 = f\left(x_k + \Delta t\, k_3\right)$$

$$x_{k+1} = x_k + \tfrac{\Delta t}{6}\left(k_1 + 2k_2 + 2k_3 + k_4\right)$$

Here $f(x)$ is short for $f(x, u_k, \theta)$, with the inputs held at their values during the step. The values k_1 to k_4 are rates of change at the start, twice in the middle and at the end of the step. Their names are customary and not related to the step index k. On the highway trip of this section, Euler with steps of one second stays within 0.002 degrees of RK4, and Euler with the 5-second steps of the animations within 0.012 degrees. Day 2 explains and measures the difference.

The parts of the state change at very different speeds. At 40 A a full pack is empty after $60 / 40 = 1.5$ hours. The temperature responds within one time constant, 11 to 68 minutes depending on the fan. The

capacity fades over months: One fifth of its 60 Ah is 12 Ah, and at 0.049 Ah per trip the pack has lost them after 12 / 0.049 = 245 trips. These time scales make the pack a good teacher for every later choice of the course.

The first result of the project is a simulated trip of two hours on the highway profile, in steps of one second, in air of 35 degrees and with the fan off. The pack starts at a state of charge of 0.95 and at 36 degrees.

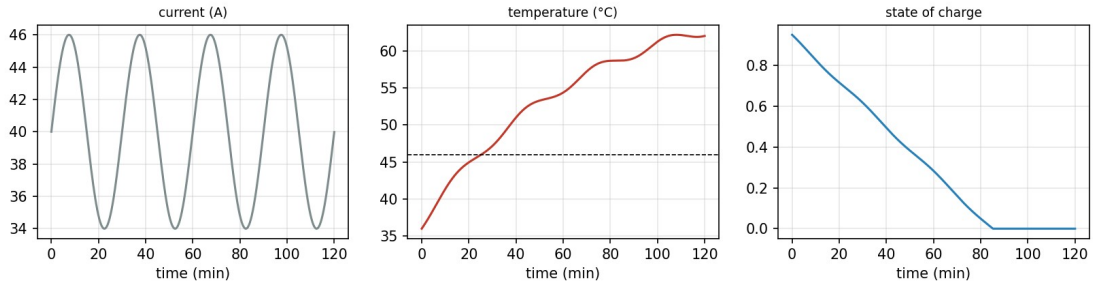


Figure. Two hours of a highway trip at 35 degrees outside: current, temperature and state of charge of the simulated pack without cooling. The dashed line marks the limit of 46 degrees.

In the left panel the current swings four times between 34 and 46 A. In the middle panel the temperature crosses the limit of 46 degrees after 25 minutes and reaches its peak of 62.2 degrees after 108 minutes. It is still on its way to the steady state near 68 degrees, and its small waves follow the current. In the right panel the state of charge falls from 0.95 to zero within 85 minutes, as 0.95 × 60 / 40 = 57 / 40 = 1.425 hours, or 85.5 minutes, predicts. The simulation then holds it at zero and keeps drawing the current, because the model has no rule that ends a trip. The last 35 minutes therefore show the thermal behaviour alone.

> **Python code in the Colab notebook, Section 1.**
>
> Open Section 1 of the Colab notebook and run it. The cell calls the function `simulate` for a trip of two hours in steps of one second. Section 0, the setup, defines this function together with `pack_derivatives`, the model of the pack with the rates of change given above, and `rk4_step`, which advances the model by one step. The cell plots the current, the temperature and the state of charge and prints the peak temperature and the capacity lost on the trip.

> Check your understanding (2). The pack delivers a constant current of 30 A with the fan off, in air of 35 degrees. With R = 0.045 ohm and hA_0 = 2.2 W/K, at which temperature does it settle?
>
> A. About 40.5 degrees
>
> B. About 53.4 degrees
>
> C. About 67.7 degrees
>
> D. About 75.5 degrees

What each arrow buys

The project now builds the three counterparts next to the same simulated pack. The virtual side starts from datasheet values, which differ a little from the real pack. A datasheet is the document in which a manufacturer lists the nominal values of a product. The real pack has the parameters of the table above. The datasheet states a thermal mass of 10200 instead of 9000 J/K, a passive conductance of 2.9 instead of

2.2 W/K and a resistance of 0.038 instead of 0.045 ohm. The parameters of the virtual side are written $\hat{\theta}$, again with a hat. Control engineering calls the asset that is controlled the plant.

Each counterpart advances its belief with the same model as the pack, but with its own parameters:

$$\hat{x}_{k+1} = \text{step}\left(\hat{x}_k,\, u_k,\, \hat{\theta}\right)$$

Here step stands for one time step of the model, by the Euler rule or by RK4. This part of the work is called the prediction, and the three levels differ in what they do besides it.

The digital model only predicts. It starts from the same state as the pack and never hears from it again, which is called running in open loop. With a thermal mass and a conductance that are both too large, the pack of the model warms too slowly.

The first arrow. The digital shadow listens to a temperature sensor, and a sensor never returns exactly the true value. The reading is

$$y_k = T(t_k) + b + \varepsilon_k, \qquad \varepsilon_k \sim \mathcal{N}(0, \sigma^2)$$

The bias b is a constant offset in every reading, 0.2 degrees for the sensor of the pack. The noise ε_k, the Greek letter epsilon, is a random error that is new in every reading. The sign $\sim$ reads *is distributed as*, and $N(0, \sigma^2)$ is the normal distribution with mean 0 and standard deviation σ, the Greek letter sigma, here 0.35 degrees. About two thirds of the noise values lie between $-\sigma$ and $+\sigma$. The sensor then rounds the reading to the nearest multiple of 0.25 degrees, the resolution of the analogue-to-digital converter (ADC) that turns the signal of the sensor into a number. With a probability of 1 percent the reading is lost on the way, which is called dropout or packet loss. A drift, an offset that grows with time, is also available. It is zero here and returns in the daily task.

> **In everyday life: bias and noise**
>
> A kitchen scale that shows 20 grams with nothing on it has a bias, which its tare button removes. A bathroom scale that shows a slightly different weight every time the same person steps on it has noise, and the mean of several readings lies closer to the truth than one reading. A clock that runs two minutes fast has a bias, and a clock that gains a few seconds every day has a drift.

Every two minutes the shadow replaces its temperature estimate by the latest reading:

$$\hat{T} \leftarrow y_k$$

The arrow reads *is replaced by*. The time between two such updates is the synchronisation interval, written Δt_{sync} and equal to 120 seconds here. Between two updates the shadow predicts like the model.

The second arrow. The digital twin does everything the shadow does and also sets the fan. Before every time step it computes the fan duty from its own estimate:

$$u = \text{clip}\left(\frac{\hat{T} - 42}{6},\; 0,\; 1\right)$$

The function clip(z, 0, 1) returns z when it lies between 0 and 1, returns 0 for smaller values and 1 for larger ones. The fan therefore starts at an estimate of 42 degrees, runs at half speed at 45 degrees and at full speed from 48 degrees on. A rule whose output grows in proportion to the distance from a threshold is called a proportional controller. Because the estimate decides the input and the input changes the pack, the twin is said to close the loop.

Worked example: one reading and the fan duty it causes

The pack is at 44.13 degrees when a reading is due, and the noise of this reading is −0.41 degrees. The sensor returns 44.13 + 0.2 − 0.41 = 44.33 − 0.41 = 43.92 degrees. The converter rounds it to the nearest multiple of 0.25: 43.92 / 0.25 = 175.68, which rounds to 176, and 176 × 0.25 = 44.00 degrees. The shadow and the twin now believe 44.00 degrees, and the error of the belief is 44.00 − 44.13 = −0.13 degrees. The twin sets the fan to (44.00 − 42) / 6 = 2.00 / 6 = 0.33.

The quality of a belief is measured against the true temperature, which is known because the pack is simulated. The error at step k and its summary over a run of N steps are

$$e_k = \hat{T}_k - T_k, \qquad \text{RMSE} = \sqrt{\frac{1}{N}\sum_{k=1}^{N} e_k^2}$$

The summary is the root mean square error, abbreviated RMSE. The sign Σ means: Add the term behind it for $k = 1, 2, \ldots, N$. The errors are squared, their mean is taken, and the root brings the result back to degrees. Squaring removes the signs, so that positive and negative errors cannot cancel, and it gives large errors more weight. The largest error, $\max_k |e_k|$, is reported as well.

Worked example: a root mean square error by hand

Four errors of 0.2, −0.4, 0.1 and −0.3 degrees have the squares $0.2^2 = 0.04$, $(-0.4)^2 = 0.16$, $0.1^2 = 0.01$ and $(-0.3)^2 = 0.09$. Their sum is 0.04 + 0.16 + 0.01 + 0.09 = 0.30, their mean is 0.30 / 4 = 0.075, and the RMSE is √(0.075) = 0.27 degrees. The plain mean of the four errors is (0.2 − 0.4 + 0.1 − 0.3) / 4 = −0.4 / 4 = −0.1 degrees, much smaller in size, because positive and negative errors cancel in it.

The model, with no data, drifts away from the truth by several degrees. The shadow corrects its belief with a reading every two minutes and follows the pack closely, yet the pack still overheats, because nothing acts on it. Only the twin, which also switches the fan, keeps the pack below its limit of 46 degrees. The table gives the results for the trip of two hours.

Measure	Digital model	Digital shadow	Digital twin
Error of the belief, RMSE	3.60 °C	0.32 °C	0.36 °C
Largest error of the belief	5.55 °C	1.06 °C	1.19 °C
Peak temperature of the pack	62.2 °C	62.2 °C	45.1 °C
Time of the pack above 46 °C	95 minutes	95 minutes	none
Capacity lost in the trip	0.049 Ah	0.049 Ah	0.034 Ah
Trips until 80 percent of the capacity is left	245	245	354

The two arrows buy different things. The first arrow reduces the error of the belief from 3.60 to 0.32 degrees, by (3.60 − 0.32) / 3.60 = 3.28 / 3.60 = 0.91, or 91 percent, to about the size of the sensor's own error, and it leaves the pack unchanged. The second arrow changes the asset: The peak falls by 62.2 − 45.1 = 17.1 degrees, and the pack never crosses its limit. The error of the twin is measured against the cooler pack that it controls, which explains its small difference from the shadow.

The last two rows follow from the ageing law. A pack is commonly retired when its capacity has fallen to 80 percent of the new value, that is, when it has lost 0.2 Q_0. If every trip costs the same loss ΔQ, the number of trips until then is

$$n_{80} = \frac{0.2\, Q_0}{\Delta Q}$$

Without control this gives 0.2 × 60 / 0.0489 = 12 / 0.0489 = 245 trips, and with the twin 12 / 0.0339 = 354 trips. Since 354 / 245 = 1.45, the twin gives the pack 45 percent more trips. The cooler pack ages more slowly: Its Arrhenius factor is 2.1 on average instead of 3.1. The number is a projection and not a measurement, because it assumes that all trips are alike.

> **Worked example: where the twin holds the pack**
>
> The fan rule and the energy balance together fix the temperature at which the controlled pack settles. In the steady state the cooling equals the heating of 72 W, $(2.2 + 12\ u) \times (T - 35) = 72$, and the fan rule demands $u = (T - 42) / 6$. Inserting the fan rule gives $12\ u = 2\ (T - 42) = 2T - 84$. The conductance is then $2.2 + 2T - 84 = 2T - 81.8$, and the balance becomes $(2T - 81.8)(T - 35) = 72$. Multiplied out, this is $2T^2 - 70T - 81.8T + 2863 = 72$, which simplifies to $2T^2 - 151.8T + 2791 = 0$. Divided by 2, it reads $T^2 - 75.9T + 1395.5 = 0$.
>
> An equation $T^2 + pT + q = 0$ has the roots $T = (-p \pm \sqrt{(p^2 - 4q)}) / 2$. Here $p = -75.9$ and $q = 1395.5$, so $p^2 - 4q = 5760.81 - 5582 = 178.81$, and $\sqrt{(178.81)} = 13.372$. The root $(75.9 - 13.372) / 2 = 31.26$ lies below 42 degrees, where the rule keeps the fan off, so it does not describe the controlled pack. The other root is $T = (75.9 + 13.372) / 2 = 89.272 / 2 = 44.64$ degrees, with the fan duty $u = (44.64 - 42) / 6 = 2.64 / 6 = 0.44$.
>
> A check confirms the result: The conductance is $2.2 + 12 \times 0.44 = 2.2 + 5.28 = 7.48$ W/K, and the cooling is $7.48 \times (44.64 - 35) = 7.48 \times 9.64 = 72.1$ W, the heating of 72 W up to rounding.
>
> In the simulation the controlled pack stays between 43.5 and 45.1 degrees after the first twenty minutes, and the fan runs at a mean duty of 0.43 in that time. The pack moves around the steady state because the current swings.

The first arrow buys knowledge; the second arrow buys a better asset, and it is the one that carries the risk. The risk is that a wrong belief now changes the asset, and the scenarios further below test this.

Animation on the lecture page: The pack and three virtual counterparts over two hours, computed in the browser with Euler steps of 5 seconds. Press play and watch the errors and the peak temperatures.

At the end of the two hours the readouts of the animation show a model error of 8.86 degrees and a shadow error of 0.25 degrees. The peaks are 62.2 degrees without control and 45.2 degrees with the twin. The model error is larger than in the table above, where the model ends 5.5 degrees too cold, and the reason is the resistance. In the table all counterparts start with the true resistance of 0.045 ohm. The animation and the loop of the reference architecture below use the datasheet value of 0.038 ohm for the virtual side.

> **Interactive lab, Part C.**
>
> The pack and one virtual counterpart of your choice in the browser, a digital model, a shadow or a twin, with sliders for the synchronisation interval and for the bias, the noise and the losses of the sensor. A scoreboard collects the error of the belief, the peak of the pack and the time above the limit, truly and as reported, for every run. Part C opens at https://utkukose.github.io/digital-twin-veltech-NB-lecture/days/day-01/lab.html#pc.

Python code in the Colab notebook, Section 2.

Open Sections 2 and 3 of the Colab notebook and run them. The function `run_ladder` runs the model, the shadow and the twin next to the same simulated pack: `read_sensor` produces the readings, the line `x_shadow[1] = T_meas` is the synchronisation, and `cooling_policy` is the decision that only the twin sends back. In Section 3 the function `report` prints the errors and the peak temperatures of the three.

Check your understanding (3). The shadow follows the pack to a fraction of a degree, yet the pack overheats. Why?

A. The sensor is broken

B. A shadow has no automatic path back to the asset, so it cannot change it

C. The model is too simple

D. Shadows always overheat

How often should the twin listen?

Synchronising more often seems always better, but it is not. Between updates the belief drifts because the model is not perfect. At every update the belief copies the reading, including its noise and bias. Two errors therefore meet in the belief, and the synchronisation interval decides how they combine. In the experiment of this section the error is smallest at an interval in between, and that interval should be chosen from a measurement, as the animation and the table below show.

In everyday life: prediction between updates

A navigation app knows the position of a car from satellite fixes. In a tunnel the fixes stop, and the app moves the car along the road at its last known speed, a prediction with a model. The longer the tunnel, the further this estimate can drift from the true position, and the first fix after the tunnel corrects it at once. A fix every second would not help much either, because every fix has an error of a few metres of its own.

A delivery app that follows a courier shows the same trade-off. The courier rides 3000 m at 4 m/s and waits 40 s at each of four traffic lights, at 600, 1300, 2000 and 2600 m, so the ride takes $3000 / 4 + 4 \times 40 = 750 + 160 = 910$ seconds. Every fix of the phone carries the position, with an error of standard deviation 5 m, and the speed. Between two fixes the app moves the marker on at the speed of the last fix, a prediction with a model of the ride.

Animation on the lecture page: A courier and the marker of a delivery app, computed in the browser. Choose the time between two fixes and play the trip: The map shows the courier and the marker, and the chart below it shows the error of the marker over the trip.

With a fix every 10 seconds the phone sends 92 fixes, and the RMSE of the marker over the trip is 5.5 m, close to the 5 m of a single fix. With a fix every 120 seconds it sends 8 fixes, and the RMSE grows to 105 m, with a largest error of 391 m. The large errors arise at the lights. After a fix taken while the courier rides, the marker runs on through a red light, and after a fix taken while the courier waits, the marker stays behind when the courier rides on. The error of a single fix sets a floor that frequent fixes cannot remove,

and the changes of speed that the model does not know make the error grow with the interval. The battery pack follows the same pattern, with the drift of the model in the place of the changes of speed.

Right after an update the belief has the error of the reading, $b + \varepsilon_k$. Then the model runs alone. Because its parameters are wrong, its temperature moves away from the true one at a rate v, the drift rate of the model, in degrees per second. A time s after the update, the error is about

$$e(s) \approx b + \varepsilon_k + v\,s, \qquad 0 \le s < \Delta t_{\text{sync}}$$

The sign ≈ reads *is approximately*, because the drift rate changes slowly during a trip. It is the difference of two energy balances at the same temperature, the datasheet model minus the pack:

$$v = \frac{I^2 R - \widehat{hA}_0\,(T - T_{\text{amb}})}{\widehat{mc_p}} - \frac{I^2 R - hA_0\,(T - T_{\text{amb}})}{mc_p}$$

The hats mark the datasheet values. At 40 A and 50 degrees with the fan off, the heating is 72 W in both terms. The datasheet model loses 2.9 × 15 = 43.5 W to the air and keeps 72 − 43.5 = 28.5 W, and the pack loses 2.2 × 15 = 33 W and keeps 72 − 33 = 39 W. Hence v = 28.5 / 10200 − 39 / 9000 = 0.00279 − 0.00433 = −0.00154, about −0.0015 degrees per second, or −0.0015 × 60 = −0.09 degrees per minute: The belief falls behind the pack. Averaging the square of the error over one interval and over the noise gives the error of the shadow as a function of the interval:

$$\text{RMSE}(\Delta t_{\text{sync}}) \approx \sqrt{\sigma^2 + b^2 + b\,v\,\Delta t_{\text{sync}} + \tfrac{1}{3}\,v^2\,\Delta t_{\text{sync}}^2}$$

Each term has a meaning. The first two belong to the sensor: However often the shadow listens, its error cannot fall below $\sqrt{(\sigma^2 + b^2)}$, which is $\sqrt{(0.35^2 + 0.2^2)} = \sqrt{(0.1225 + 0.04)} = \sqrt{(0.1625)} = 0.40$ degrees here. The last term belongs to the model and makes the error grow with the interval. The third term couples the two. It is negative when bias and drift have opposite signs, as they have here: The sensor reads too high and the model runs too cold, so the two errors cancel in part. The curve then dips before it rises, with its lowest point at $\Delta t_{\text{sync}} = -3b / (2v)$. With an unbiased sensor, or with a bias of the same sign as the drift, there is no dip. The formula treats the drift rate as constant, which is acceptable for intervals of up to about twenty minutes.

Worked example: the error of the shadow for two intervals

With σ = 0.35, b = 0.2 and v = −0.0015, an interval of 120 seconds lets the model drift by $v\,\Delta t_{\text{sync}}$ = −0.0015 × 120 = −0.18 degrees. The four terms under the root are $\sigma^2 = 0.35^2 = 0.1225$, $b^2 = 0.2^2 = 0.04$, $b\,v\,\Delta t_{\text{sync}} = 0.2 \times (-0.18) = -0.036$ and $1/3\,v^2\,\Delta t_{\text{sync}}{}^2 = 1/3\,(-0.18)^2 = 0.0324 / 3 = 0.0108$. Their sum is 0.1225 + 0.04 − 0.036 + 0.0108 = 0.1373, and the error is √(0.1373) = 0.37 degrees.

For 1200 seconds the drift is −0.0015 × 1200 = −1.8 degrees, and the terms are 0.1225, 0.04, 0.2 × (−1.8) = −0.36 and $(-1.8)^2 / 3 = 3.24 / 3 = 1.08$. Their sum is 0.1225 + 0.04 − 0.36 + 1.08 = 0.8825, which gives √(0.8825) = 0.94 degrees. The lowest point of the formula lies at $-3b / (2v) = -3 \times 0.2 / (2 \times (-0.0015))$ = −0.6 / (−0.003) = 200 seconds. There the drift is −0.0015 × 200 = −0.3 degrees, the last two terms are 0.2 × (−0.3) = −0.06 and $(-0.3)^2 / 3 = 0.09 / 3 = 0.03$, the sum is 0.1225 + 0.04 − 0.06 + 0.03 = 0.1325, and the error is √(0.1325) = 0.36 degrees.

Interactive lab, Part D.

The formulas of this section solved step by step with numbers: where the drift rate comes from, the drift within one interval and the RMSE of the shadow, term by term. Sliders set the noise, the bias, the drift rate and the interval. A figure draws the whole curve on a logarithmic axis and marks the chosen interval and the lowest point. Part D opens at https://utkukose.github.io/digital-twin-veltech-NB-lecture/days/day-01/lab.html#pd.

A simulation of the trip measures this curve with eight intervals. The table sets its values beside the formula and adds the peak of the pack that the twin controls.

Synchronisation interval	RMSE of the shadow, measured	RMSE by the formula	Peak of the pack with the twin
10 seconds	0.42 °C	0.40 °C	45.1 °C
30 seconds	0.38 °C	0.39 °C	45.1 °C
1 minute	0.32 °C	0.38 °C	45.1 °C
2 minutes	0.36 °C	0.37 °C	45.2 °C
5 minutes	0.40 °C	0.37 °C	45.2 °C
10 minutes	0.60 °C	0.50 °C	45.6 °C
20 minutes	0.96 °C	0.94 °C	45.5 °C
1 hour	2.37 °C	beyond the range of the formula	46.1 °C

The measured curve is a shallow U. On the left it starts at the floor that the sensor sets, and on the right it climbs steeply, to 0.60 degrees at ten minutes and 2.37 degrees at one hour. From ten seconds to five minutes all values lie between 0.32 and 0.42 degrees. The lowest value of this run is at one minute, while the formula places the lowest point near three minutes. The two do not contradict each other. Every run draws new noise, which shifts each value in the flat part by a few hundredths of a degree. In repeated sweeps the lowest value falls most often on two or five minutes. The result that holds in every run is the one that matters for a design. With this sensor, listening more often than every few minutes buys no accuracy, and listening less often than every ten minutes loses it quickly.

For the asset the interval matters less than for the belief. With the twin in control, the peak of the pack stays between 45.1 and 45.6 degrees for all intervals up to twenty minutes. It passes the limit only with one reading per hour, at 46.1 degrees. The floor of the sensor is a property of overwriting, which copies every reading in full, and Day 3 replaces overwriting by a filter that takes over only a part of each reading.

Animation on the lecture page: The RMSE of the shadow against the synchronisation interval, computed in the browser with the model and the sensor of the day. Move the slider through the eight intervals.

The horizontal axis of the animation is logarithmic. It shows $\log_{10}$ of the interval, which is 1 for 10 seconds, 2 for 100 seconds and 3 for 1000 seconds, so that short and long intervals fit on one axis. The animation uses Euler steps of 5 seconds and its own random numbers. Its values lie between 0.37 and 0.40 degrees up to two minutes and reach 0.81 degrees at ten minutes and 3.97 degrees at one hour. They climb faster than in the table, because the model of the animation also uses the datasheet resistance and drifts at about −0.0026 degrees per second.

Python code in the Colab notebook, Section 4.

Open Section 4 of the Colab notebook and run it. The cell calls `run_ladder` with eight values of `sync_every`, from ten seconds to an hour, prints the RMSE of the shadow for each and draws the curve on a logarithmic axis.

A reference architecture

A reference architecture is a plan that names the parts of a system and the responsibility of each part, without prescribing a product. Reference architectures for digital twins agree on five parts [3, 4]: the physical entity with its sensors and actuators, the connection that carries data both ways, the virtual entity that holds the model of this asset, a data store that keeps every reading and command, and the services that people and other systems use, such as monitoring, alarms and control. In the project each part becomes a small program of its own, and one loop connects them. An actuator is a device that changes the asset on command, here the fan. The parts are also called components or dimensions.

Part	Responsibility
Physical entity	The asset with its sensors and actuators
Connection	Carries readings to the virtual side and commands back, and may distort, delay or lose them
Virtual entity	The model of this asset and its belief
Data store	Every reading, belief and command with its time
Services	What people and other systems use: monitoring, alarms, prediction and control

Interactive lab, Part E.

Six responsibilities in the twin of the pack, each to be assigned to one of the five parts of the reference architecture. Part E opens at https://utkukose.github.io/digital-twin-veltech-NB-lecture/days/day-01/lab.html#pe.

One loop runs the parts in a fixed order, once per second of simulated time, and this order is the algorithm of the twin. First the pack moves one time step. The connection then checks whether a reading is due. The virtual entity predicts one step, with the same current and fan duty as the pack, and corrects its belief if a reading has arrived. The services are evaluated on the belief. If the twin has authority, the permission to act, the fan duty is sent to the pack and takes effect in the next step. Last, the true values, the belief and the command of this second are stored. The loop ends after 7200 steps, and it would stay the same if the simulated pack were replaced by a real one.

The correction is written more generally than the overwriting of the shadow:

$$\hat{T} \leftarrow \hat{T} + K\left(y_k - \hat{T}\right)$$

The difference between the reading and the prediction, $r_k = y_k - \hat{T}$, is called the residual. The gain K, a number between 0 and 1 without a unit, is the share of the residual that the belief takes over. With $K = 1$, the value used so far, the belief becomes the reading, as in the shadow. With $K = 0$ the reading is ignored, as in the digital model. For a belief of 44.6 degrees and a reading of 45.0 degrees, $K = 0.5$ gives $44.6 + 0.5 \times (45.0 - 44.6) = 44.6 + 0.5 \times 0.4 = 44.6 + 0.2 = 44.8$ degrees. A gain of 0.5 lowers the error of the belief

from 0.29 to 0.21 degrees, because only half of the error of each reading enters the belief. Day 3 computes a suitable gain from the noise of the sensor and the quality of the model.

Service	Rule	Meaning
`fan_duty`	$u = \mathrm{clip}((T - 42) / 6, 0, 1)$	Command for the fan, the rule of the twin
`overheat`	$T > 46$	Alarm: true while the belief is above the limit
`range_min`	$60 \times \mathrm{SoC} \times Q / I$	Minutes until the pack is empty at the present current
`health_pct`	$100 \times Q / Q_0$	Usable capacity in percent of the new value, the state of health

All four rules use the belief and not the true state, which the services cannot see. At the start of the trip the remaining range is 60 × 0.95 × 60 / 40 = 60 × 57 / 40 = 60 × 1.425 = 85.5 minutes, the moment at which the state of charge reaches zero in the figure of the trip. After two hours with the twin, the health is 100 × 59.966 / 60 = 100 × 0.99943 = 99.94 percent.

With the sensor of the previous section and one reading every 120 seconds, the loop receives 60 readings in two hours. It keeps the peak of the pack at 45.2 degrees, and its belief has an RMSE of 0.29 degrees. The loop makes two simplifications. Only the path from the asset to the virtual side passes through the connection, while the command reaches the fan directly, without loss or delay. The data store is a list in the memory of the program to which records are only added, which is called append-only.

> **Python code in the Colab notebook, Section 5.**
>
> Open Section 5 of the Colab notebook and run its two cells. In the first cell each of the five parts of the architecture is a small Python class, and the line `self.x[1] += gain * (obs["T"] - self.x[1])` is the correction of the belief. The second cell connects the parts in the function `orchestrate`, one loop of `phys.step`, `conn.poll`, `virt.step`, `virt.correct`, `svc.evaluate`, `phys.actuate` and `store.put`. It runs the loop for two hours and prints one row for every 15 minutes, the RMSE of the belief and the number of readings.

> Check your understanding (4). Where does the decision to switch on the fan live in the reference architecture?
>
> A. In the physical entity
>
> B. In the data store
>
> C. In the services, which act through the connection
>
> D. In the sensor

Testing before hardware exists

Many industrial twins are built against a simulated asset first [5]. A simulated asset can be made to fail on purpose, its true state is known, and nothing breaks when the controller is wrong. The project uses this to grade the twin under faults that are hard to stage on real hardware. The true state of the asset is called the ground truth, and making a part fail on purpose is called fault injection.

> **In everyday life: testing on a simulated asset**
>
> Pilots practise engine failures in flight simulators, where an engine can be made to fail on purpose and nobody is at risk. Car makers test airbags with crash-test dummies before any person sits in a new car. In both cases the failure is staged where the truth is known and nothing valuable breaks, as in the scenarios of this section.

Grading compares what happened with what the twin reported. For every scenario the grading counts the time in which the pack was truly above its limit and the time in which the twin believed so:

$$t_{\text{true}} = \Delta t \sum_{k=1}^{N} [\, T_k > 46 \,]$$

$$t_{\text{rep}} = \Delta t \sum_{k=1}^{N} \left[\, \hat{T}_k > 46 \,\right]$$

A square bracket around a condition counts 1 when the condition is true and 0 when it is false. With time steps of one second, both sums are numbers of seconds, and for a twin that reports honestly they agree. The loop of the architecture runs under five scenarios.

Scenario	What is changed	RMSE of the belief	Peak of the pack	Truly above 46 °C	Reported above 46 °C
`nominal`	Nothing	0.29 °C	45.2 °C	0 s	0 s
`sensor bias -5 C`	The sensor reads 5 degrees too low	5.00 °C	49.2 °C	5687 s	0 s
`70% packet loss`	Seven of ten readings are lost	0.53 °C	45.3 °C	0 s	0 s
`slow link (20 min)`	One reading every 20 minutes	1.01 °C	46.3 °C	262 s	98 s
`no authority`	The twin may not set the fan	0.31 °C	62.2 °C	5687 s	5640 s

One result stands out: When the temperature sensor reads five degrees too low, the pack stays above its limit for over an hour while the twin, which trusts its sensor, reports no overheating at all. The counts are 5687 seconds, 5687 / 60 = 95 minutes, against 0 seconds.

The formulas of the twin explain this result. After every update the belief is about $T + b$, so the twin applies its fan rule and its alarm to a shifted temperature:

$$u = \operatorname{clip}\left(\frac{T + b - 42}{6},\ 0,\ 1\right), \qquad \text{alarm when } T + b > 46$$

In true degrees the fan starts at $42 - b$ and the alarm sounds at $46 - b$. With $b = -5$ the fan starts at 47 degrees, which is already above the limit, and the alarm would need 51 degrees. The pack settles between the two, too hot and never reported. The worked example on where the twin holds the pack, repeated with the shifted rule $u = (T - 47) / 6$, gives the conductance $2.2 + 2T - 94 = 2T - 91.8$ and the equation $T^2 - 80.9T + 1570.5 = 0$. Its larger root is $T = (80.9 + \sqrt{(6544.81 - 6282)}) / 2 = (80.9 + 16.211) / 2 = 48.56$ degrees, with the fan duty $(48.56 - 47) / 6 = 1.56 / 6 = 0.26$. The fan does run in the simulation, at a mean duty of 0.19, and it lowers the peak from 62.2 to 49.2 degrees. The time above the limit is nevertheless the

same 5687 seconds as in the scenario without authority, because the cooling starts only after the limit has been passed.

The other scenarios are milder. With 70 percent packet loss only 3 of 10 readings arrive, one every 120 / 0.3 = 400 seconds on average, the error of the belief grows to 0.53 degrees, and the pack still stays below its limit. With one reading every 20 minutes the pack is above the limit for 262 seconds, and the twin reports 98 seconds. Without authority the twin is a shadow: It reports 5640 hot seconds, close to the true 5687, and can do nothing about them.

A biased sensor controls the twin, which is why the security of the sensing chain is part of the architecture [7]. The sensing chain is everything between the asset and the belief: the sensor, the converter, the network and the software that passes the reading on. Whoever can shift a reading in this chain can steer the fan. The twin of this day cannot notice the bias, because after every overwrite its belief agrees with the sensor. A check needs a second source that the sensor cannot influence, such as a second sensor or a prediction that is never overwritten. A twin with such a check can give up its authority when the two sources disagree, and Day 3 builds one.

A twin tested only against a simulator is, of course, tested only against the assumptions of that simulator. Testing a twin against measurements of the real asset is called validation, and it remains necessary when the hardware exists.

> **Python code in the Colab notebook, Section 6.**
>
> Open Section 6 of the Colab notebook and run it. The function `scenarios` runs the loop of Section 5 under five conditions, among them a biased sensor, `bias=-5.0`, a lossy connection, `dropout=0.70`, and a twin without authority, `authority=False`. For each it prints the seconds in which the pack was truly too hot, `truly_hot_s`, next to the seconds in which the twin said so, `twin_says_hot_s`.

> Check your understanding (5). Under a sensor bias of minus five degrees, the twin reports no overheating while the pack overheats. What is the root cause?
>
> A. The fan is too weak
>
> B. The twin trusts a reading it has no way to check
>
> C. The model is too accurate
>
> D. The battery is new

> Check your understanding (6). A sensor reads 3 degrees too low, and the twin overwrites its belief with every reading. At which true temperature does this twin report overheating, with a limit of 46 degrees?
>
> A. At 43 degrees
>
> B. At 46 degrees
>
> C. At 49 degrees
>
> D. At 52 degrees

The twin of the pack, stage 1: a twin that keeps the pack below its limit

The methods of this day now come together in the first stage of the project. The pack has a model with four states, a sensor with bias, noise and losses, and a virtual counterpart that starts from the datasheet.

One loop of five parts predicts, corrects and acts once per second, and every result is graded against the true state of the simulated pack. The figure shows the loop of this stage with its numbers, and the table collects what the stage has shown.

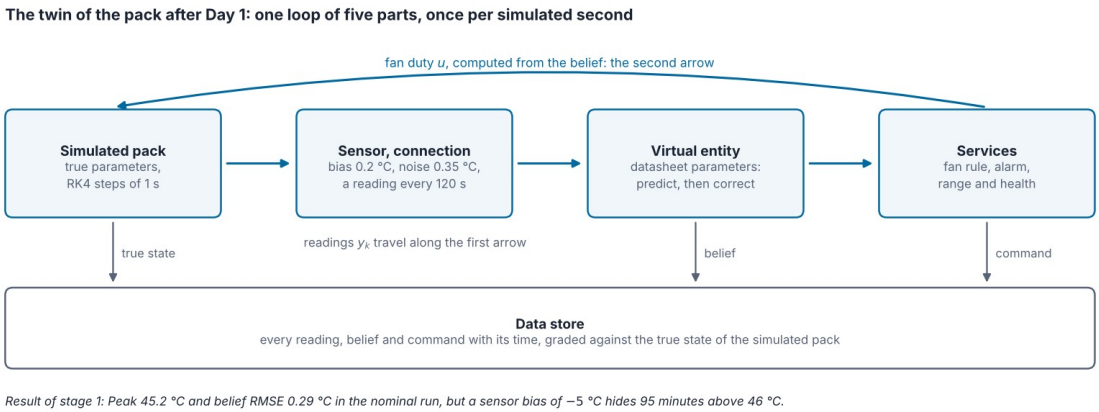


Figure. The twin of the pack after Day 1. The readings travel along the first arrow and the fan duty along the second, and the data store keeps the record against which the twin is graded.

Question	Answer of stage 1	Section
How far is the belief from the truth?	RMSE 3.60 °C for the model, 0.32 °C for the shadow and 0.36 °C for the twin	What each arrow buys
What does the second arrow change?	Peak 45.1 instead of 62.2 °C, no time above 46 °C, 354 instead of 245 trips	What each arrow buys
How often should the twin listen?	Every one to five minutes, while intervals above ten minutes cost accuracy	How often should the twin listen?
Does the loop of five parts work?	60 readings in two hours, peak 45.2 °C, RMSE 0.29 °C	A reference architecture
Where does it fail?	A sensor bias of −5 °C hides 95 minutes above the limit	Testing before hardware exists

Two weaknesses of this stage set the work of the next days. Every value of the twin comes from a numerical integration whose accuracy has not yet been measured, and Day 2 tests this simulation core. The belief copies every reading and so inherits the noise and the bias of the sensor. Day 3 therefore calibrates the model on measurements and replaces the overwriting by a Kalman filter.

Your asset

The same ideas apply to any asset with a state that can be measured and an input that can be controlled. The application of the day replaces the battery pack by a server rack, the winding of an electric motor or the air in a greenhouse. Each has its own heat input, cooling and limit, and the model, the shadow and the twin are built for it in the same way.

The four assets share one model. It is the energy balance of the pack with a general heat input $P(t)$ in watts and a heat capacity C in joules per kelvin:

$$\frac{dT}{dt} = \frac{P(t) - (hA_0 + hA_1 u)\,(T - T_{\text{amb}})}{C}$$

For the battery pack, $P(t) = I(t)^2 \, R$ and $C = m \, c_p$. For the other assets, u is the setting of their cooling between 0 and 1. The steady state and the time constant follow from the formulas of the pack, with P in the place of $I^2 \, R$.

Asset	C in J/K	hA_0 and hA_1 in W/K	Ambient temperature and limit in °C	Heat input $P(t)$ in W	Length of the run and time step
Battery pack	9000	2.2 and 12	35 and 46	52 to 95, following the current of the highway profile	2 hours, 5 seconds
Server rack	1.2×10^5	40 and 400	24 and 35	3000, rising to 4500 in the first half of every hour	2 hours, 10 seconds
Motor winding	4000	1.5 and 8	30 and 90	43 to 235, in a cycle of 20 minutes	1.5 hours, 5 seconds
Greenhouse air	3×10^6	500 and 4000	18 and 32	Rising to 30000 and back to 0 during the first 12 hours, then 0	24 hours, 60 seconds

The three counterparts are built in the same way for every asset, with the Euler rule. The datasheet model has a heat capacity that is 15 percent too large and a passive conductance that is 30 percent too large. The shadow overwrites its belief at intervals of one sixtieth of the run, which is 2 minutes for the pack and 24 minutes for the greenhouse. Its readings have no bias, and their noise has a standard deviation of $0.35 \times (T_{\mathrm{lim}} - T_{\mathrm{amb}}) / 11$ degrees, where T_{lim} is the limit of the asset. The twin also sets the cooling with a proportional rule that starts four degrees below the limit and reaches full duty at the limit:

$$u = \mathrm{clip}\left(\frac{\hat{T} - (T_{\mathrm{lim}} - 4)}{4}, \; 0, \; 1 \right)$$

Asset	RMSE of the model	RMSE of the shadow	Peak without control	Peak with the twin	Time above the limit, without control and with the twin
Battery pack	3.58 °C	0.28 °C	62.2 °C	44.6 °C	95 and 0 minutes
Server rack	11.52 °C	0.47 °C	100.9 °C	35.1 °C	114 and 6 minutes
Motor winding	10.42 °C	1.47 °C	110.4 °C	89.5 °C	44 and 0 minutes
Greenhouse air	6.31 °C	0.95 °C	73.3 °C	30.5 °C	642 and 0 minutes

The first row is the result for the battery pack, and the other rows are the results for the other three assets. For every asset the first arrow removes most of the error of the belief, and the second arrow removes most or all of the time above the limit. The server rack is the hard case. At full cooling its conductance is 40 + 400 = 440 W/K, and at peak load its steady state is 24 + 4500 / 440 = 24 + 10.23 = 34.2 degrees, only 35 − 34.2 = 0.8 degrees below its limit, and the proportional rule reaches full duty only at the limit itself. The rack therefore spends 6 minutes just above 35 degrees.

A parcel on its way to a customer is an asset of daily life with a state that can be measured. The delivery app follows the courier of the section on synchronisation with fixes from a phone, as the shadow follows the pack with readings, and it faces the same two choices: how often to listen and how to predict between two updates.

> **Interactive lab, Part F.**
>
> The delivery tracker: the courier with an interval between fixes and a prediction rule of your choice. Every run enters a table, and the part asks for the longest interval that keeps the RMSE of the marker below 20 metres and for two positions of the marker by hand. Part F opens at https://utkukose.github.io/digital-twin-veltech-NB-lecture/days/day-01/lab.html#pf.

> **Python code in the Colab notebook, Section 7.**
>
> Open Section 7 of the Colab notebook. Its first line is a switch, `ASSET = "battery"`. Replace the word by `server_rack`, `motor` or `greenhouse` and run the section. The function `run_asset` simulates the asset with the Euler rule and builds its model, its shadow and its twin, and the cell prints their errors and the time above the limit.

Going further (optional)

How strongly a sensor bias controls the twin can be measured. A bias that reads low hides real overheating, and a bias that reads high reports overheating that never happened. Both are failures of trust, and Day 3 adds an independent physics check that detects them.

Sensor bias *b*	Fan starts at a true temperature of	Alarm sounds at a true temperature of	Peak of the pack	Truly above 46 °C	Reported above 46 °C
−6 °C	48 °C	52 °C	50.1 °C	5687 s	0 s
−4 °C	46 °C	50 °C	48.4 °C	5687 s	0 s
−2 °C	44 °C	48 °C	46.8 °C	3159 s	0 s
0 °C	42 °C	46 °C	45.4 °C	0 s	0 s
+2 °C	40 °C	44 °C	44.0 °C	0 s	171 s
+4 °C	38 °C	42 °C	42.9 °C	0 s	2236 s

The second and the third column are $42 - b$ and $46 - b$ from the formulas above, and the last three columns are measured in simulations of the trip. Two comparisons explain each row. The pack is truly too hot when its peak is above 46 degrees, and the twin reports overheating when the peak is above the alarm level of the row. With $b = -2$ the peak of 46.8 degrees lies above the limit and below the alarm level of 48 degrees, and 3159 / 60 = 53 minutes of overheating stay hidden. With $b = +4$ the pack never passes 42.9 degrees, yet the alarm level has fallen to 42 degrees, and the twin reports 2236 / 60 = 37 minutes of overheating that never happened.

Applications of the same architecture in medicine and health care are reviewed in [8, 9].

> **Python code in the Colab notebook, Section 8.**
> Open Section 8 of the Colab notebook, which is optional, and run it. The loop repeats `orchestrate` for sensor biases from minus six to plus four degrees and prints the real and the reported time above the limit. The last cell shows four sliders, for the synchronisation interval, the bias, the noise and the packet loss.

Answers to the knowledge checks

Quick check. `0.0`. The expression `(39 - 42) / 6` is negative, and `max(0.0, ...)` clips it to zero, so the fan stays off.

1. B. Data flow automatically only from the asset to the virtual side.

2. B. The heating is $30^2 \times 0.045 = 900 \times 0.045 = 40.5$ W, the rise above the air is 40.5 / 2.2 = 18.4 degrees, and 35 + 18.4 = 53.4 degrees.

3. B. Knowledge of the state changes nothing unless it can act on the asset.

4. C. Services use the virtual entity and send commands to the asset through the connection.

5. B. A twin that overwrites its belief with a biased reading acts on the wrong state.

6. C. The belief is the true temperature minus 3 degrees, so it passes 46 degrees when the pack is at 46 + 3 = 49 degrees.

References

[1] Grieves, M., & Vickers, J. (2017). Digital twin: Mitigating unpredictable, undesirable emergent behavior in complex systems. In F.-J. Kahlen, S. Flumerfelt & A. Alves (Eds.), Transdisciplinary Perspectives on Complex Systems (pp. 85-113). Springer.

[2] Kritzinger, W., Karner, M., Traar, G., Henjes, J., & Sihn, W. (2018). Digital Twin in manufacturing: A categorical literature review and classification. IFAC-PapersOnLine, 51(11), 1016-1022. https://doi.org/10.1016/j.ifacol.2018.08.474

[3] Tao, F., Xiao, B., Qi, Q., Cheng, J., & Ji, P. (2022). Digital twin modeling. Journal of Manufacturing Systems, 64, 372-389.

[4] Qi, Q., Tao, F., Hu, T., Anwer, N., Liu, A., Wei, Y., Wang, L., & Nee, A. Y. C. (2021). Enabling technologies and tools for digital twin. Journal of Manufacturing Systems, 58, 3-21.

[5] Crespi, N., Drobot, A. T., & Minerva, R. (2023). The digital twin: What and why? In The Digital Twin (pp. 3-20). Springer International Publishing.

[6] Van Rossum, G., & Drake, F. L. (2009). Python 3 Reference Manual. CreateSpace.

[7] Kose, U. (2023). Security issues in artificial intelligence use for metaverse and digital twin setups. In Digital Twin Driven Intelligent Systems and Emerging Metaverse (pp. 331-349). Springer Nature Singapore.

[8] Uysal, I., & Kose, U. (2024). Explainability and the role of digital twins in personalized medicine and healthcare optimization. In Explainable Artificial Intelligence (XAI) in Healthcare (pp. 141-156). CRC Press.

[9] Kose, U., Tükel, G. Ö., Sürücü, E. B., Karataş, M. E., & Uysal, I. (2025). Graph Neural Networks-Based Digital Twin Modeling of WBC and ANC Dynamics for Personalized Time-Series Prediction. In 2025 Medical Technologies Congress (TIPTEKNO) (pp. 1-4). IEEE.

[10] Kluyver, T., Ragan-Kelley, B., Pérez, F., Granger, B., Bussonnier, M., Frederic, J., Kelley, K., Hamrick, J., Grout, J., Corlay, S., Ivanov, P., Avila, D., Abdalla, S., Willing, C., & Jupyter development team. (2016). Jupyter Notebooks: A

publishing format for reproducible computational workflows. In F. Loizides & B. Schmidt (Eds.), Positioning and Power in Academic Publishing: Players, Agents and Agendas (pp. 87-90). IOS Press. https://doi.org/10.3233/978-1-61499-649-1-87

[11] Jones, D., Snider, C., Nassehi, A., Yon, J., & Hicks, B. (2020). Characterising the Digital Twin: A systematic literature review. CIRP Journal of Manufacturing Science and Technology, 29, 36-52.

Prof. Dr. Utku Kose

Full Professor, Department of Computer Engineering, Süleyman Demirel University, Isparta, Türkiye

Founding Director, AI Application and Research Center (YAZEM), Süleyman Demirel University

Head of the Computer Science Division, Department of Computer Engineering, Süleyman Demirel University

Additional affiliations: University of North Dakota (USA), Universidad Panamericana (Mexico City, Mexico), Vel Tech University (Chennai, India)

IEEE Senior Member, ACM Professional Member

ORCID 0000-0002-9652-6415 · utkukose.com · github.com/utkukose

utkukose@sdu.edu.tr · utku.kose@und.edu · ukose@up.edu.mx · utkukose@gmail.com

This course is updated in line with current developments in the field. First version: October 2026. Last update: October 2026.